ADR 0128 — A credential exists only before untrusted checkout

- **Status:** Accepted — implemented, mutation-proved two ways failing differently
- **Date:** 2026-09-06
- **Issue:** #680
- **Extends:** ADR 0122
- **Related:** ADR 0119 (the safety lives in the value on every outcome)

Context

ADR 0122 supplies Git-Vista's HTTPS token to a fixed credential helper through `GIT_VISTA_CREDENTIAL_TOKEN` on the spawned git process. Git's descendants inherit that variable. Clone also deliberately uses `HookMode::Run` in the Network tier, so hooks and checkout filters selected by attacker-chosen content may execute with network access.

#680 reproduced the resulting chain. With operator configuration `core.hooksPath=hooks`, a remote containing an executable tracked `hooks/post-checkout` read the complete canary from the clone process's environment. The precondition is operator-level configuration, but the program that reads the value is supplied by the remote. A filter configured globally and selected by the remote's `.gitattributes` has the same checkout-time shape.

Clone had a second independent leak. Unlike ordinary remote operations, the handler called `.output()` directly because the destination repository did not exist yet. It logged and returned raw stderr. A descendant could therefore print the inherited token and make clone fail, placing the value in the server log and the HTTP error. Closing only this output path would still leave direct network exfiltration; closing only inheritance would leave raw credentialed output available to the next caller.

There is one more inherited source than the original report named. When token resolution chooses `GIT_VISTA_GITHUB_TOKEN` or `GH_TOKEN`, that variable is in the server's ambient environment. Removing only the internal helper variable from a later command would leave the same credential available under its source name.

Decision

1. Clone is two processes with one security boundary between them

The first process runs:

```
git clone --no-checkout -- <url> <destination>
```

It retains the existing Network-tier clone policy and Git-Vista credential helper. It downloads objects, creates refs and writes repository metadata, but does not materialise the fetched tree. Checkout hooks and content filters therefore have no attacker-supplied worktree content to execute while the token exists.

After that process exits, a second Network-tier process verifies that `HEAD` exists and runs `git checkout -f`. It keeps `HookMode::Run`, filter execution, and network access. Before spawn it explicitly removes all three known credential environment names:

```
GIT_VISTA_CREDENTIAL_TOKEN
GIT_VISTA_GITHUB_TOKEN
GH_TOKEN
```

This preserves the clone feature's deliberate hook and filter behavior. It changes when checkout happens, not whether it happens or what sandbox tier it receives. The original ten-minute Drop-

guard covers both phases as one future, so timeout or client cancellation still kills the active child and removes the partial destination.

An empty remote has no `HEAD` target. `git clone --no-checkout` succeeds in that case, while `git checkout -f` would fail. The intermediate `show-ref --verify --quiet HEAD` preserves ordinary clone behavior by treating its documented status 1 as a successful empty clone; every other nonzero result is still a clone failure.

2. A credential-bearing command cannot return raw output

`network_command_with_credential` returns `CredentialedCommand`, not the general `SandboxedCommand`. This value owns a copy of the exact token used for the child. Its only production execution method captures output and removes both URL userinfo and every literal occurrence of that token from stdout and stderr before returning `Output`. It does not expose the general command type's raw `spawn` method.

The output guarantee therefore follows the value into the next credentialed call site. A handler cannot obtain raw credential-bearing output and cannot forget to call a redactor. Redaction operates on bytes, including buffers that are not valid UTF-8; an empty token is ignored rather than treated as a match at every byte boundary.

3. Credential injection also removes ambient credential sources

`SandboxedCommand::credential_env` removes `GIT_VISTA_GITHUB_TOKEN` and `GH_TOKEN` before adding the internal helper variable. The checkout builder removes those source variables and the internal variable. This makes the resolved value the only credential supplied to the transfer and prevents the credentialless phase from inheriting a second spelling of the same secret.

Rejected alternatives

Disable hooks or filters for credentialed clone

Rejected because clone's Network tier and `HookMode::Run` are deliberate product behavior. Pinning `core.hooksPath` would address the reproduced hook but not an arbitrary globally configured filter selected by `.gitattributes`. Enumerating executable Git configuration keys would move the boundary to a list that fails open when another checkout mechanism appears.

Pass the token on an inherited pipe

Rejected for this clone fix. Git must retain a capability that its helper can use later, and arbitrary descendants of that same git process inherit open descriptors unless a larger supervisor or broker design mediates them. A one-shot pipe also remains unread for a public remote that never challenges for credentials, leaving it available when checkout begins. The two-process split uses process lifetime as the boundary and needs no new long-lived secret service.

Redact only in `handlers/clone.rs`

Rejected because it repeats the defect's structure: correctness would depend on every success, error, log, and future call site remembering a function. The command value has the token and is the earliest place that can enforce the same rule for every output path.

Consequences and limits

Clone performs two additional local git commands after transfer: a cheap HEAD existence check and, for a nonempty repository, checkout. The entire operation keeps one timeout and cleanup scope. A transfer failure is already redacted by the credential-bearing value; later commands carry no Git-Vista credential and still receive ordinary URL-userinfo redaction.

This does not make `network_command_with_credential` safe for fetch, pull, or push in an existing untrusted repository. Those commands can consult repository-local credential helpers or other executable configuration before a separate checkout boundary exists. ADR 0122's blocker on those future call sites remains in force.

Proof

`clone_checkout_runs_the_hook_without_any_credential_environment` is the original canary made permanent. It creates a source repository with a tracked executable `hooks/post-checkout`, performs the credentialed `--no-checkout` transfer through the production builder, and confirms that neither content nor the marker exists. It then selects the hook with `core.hooksPath=hooks`, runs the production credentialless checkout, and proves both that the hook ran and that it observed all three credential variables as `unset`.

`credential_redaction_removes_a_bare_token_from_both_output_streams` proves exact-token removal from stdout and stderr while preserving a neighboring invalid UTF-8 byte. The existing live private-clone tests continue to exercise the credential helper and recorded remote URL.

`failure-atlas mutation_check` ran against committed HEAD `6e631a51` under run key `gv-680-clone-token-containment`. Both baselines were green and both mutations were caught:

Run	Mutation	Distinct failure
354	Remove `--no-checkout` from `clone_transfer_args`	The real tracked `post-checkout` hook ran during the credentialed tra `clone_checkout_runs_the_hook_without_any_credential_environ` failed because the transfer materialised content and created the marke
355	Return `output.stdout` / `output.stderr` directly instead of applying `redact_literal`	`credential_redaction_removes_a_bare_token_from_both_output_` failed on the canary bytes still present in stdout.

The first arm is a spawned-git lifecycle failure through the production clone command builder. The second is a pure byte-redaction failure. Neither mutation failed to build, neither survived, and neither relies on the other's assertion.